Specification-Driven Development (SDD)

Deepika Kanchi

29th october 2026

1 What is Specification-Driven Development (SDD)

Specification-Driven Development (SDD) is a software engineering approach in which a detailed specification serves as the primary source of truth for development. The specification defines the intended behavior, requirements, constraints, and acceptance conditions of a system before implementation begins. In AI-assisted engineering, the workflow typically becomes:

Specification → Code Generation → Test Generation → Documentation → Human Review

rather than:

Prompt → AI Generates Code

2 Why the Specification Becomes the Source of Truth

A prompt is simply an instruction given to an AI model. Prompts are often ambiguous, incomplete, and difficult to govern.

2.1 Problems with Prompts

- Ambiguous requirements
- Lack of version control
- Different engineers may write different prompts
- Difficult to review and audit
- Can lead to inconsistent AI outputs

Example Prompt:

> "Build a login system."

2.2 Benefits of Specifications

- Single source of truth
- Reduced ambiguity
- Easier collaboration between humans and AI
- Better traceability from requirement to code
- Improved test automation
- Can be stored and reviewed through version control systems

3 Anatomy of a Good Specification

A high-quality specification contains the following components.

3.1 Intent

Intent explains **why** the feature exists and the business value it delivers. **Example**

> Allow users to securely access their accounts while preventing unauthorized access.

3.2 Scope

Scope clearly defines what is included and excluded. **Included**

- Login
- Logout
- Password Reset

Excluded

- Social Login
- Multi-Factor Authentication (MFA)

3.3 Constraints

Constraints define technical, security, legal, and performance requirements. **Examples**

- Password must contain at least 8 characters
- Response time must be under 2 seconds
- Must comply with GDPR regulations
- Must use Microsoft Entra ID for authentication

3.4 Interfaces

Interfaces define how systems communicate. **API Example**

```
POST /api/login
```

Input

```
{ "email": "user@example.com", "password": "password123" }
```

Output

```
{ "token": "jwt_token" }
```

3.5 Acceptance Criteria

Acceptance criteria define the conditions that must be satisfied before the feature is considered complete.

1. User can log in using valid credentials.
2. Invalid credentials return HTTP 401.
3. Account locks after five failed login attempts.
4. Successful login redirects the user to the dashboard.

3.6 Test Cases

Test cases verify that each acceptance criterion works correctly.

Test Case	Expected Result
Valid email and password	Login successful
Invalid password	HTTP 401 Unauthorized
Five consecutive failures	Account locked
Empty password field	Validation error displayed

4 Example Complete Specification

Feature: User Login

Intent

Enable secure user authentication to access the application.

Scope

- Login
- Logout

Out of Scope

- Social Login

- MFA

Constraints

- Password length ≥ 8 characters
- Response time < 2 seconds

Interface

```
POST /api/login
```

Acceptance Criteria

1. Valid credentials allow login.
2. Invalid credentials return HTTP 401.
3. Account locks after five failed attempts.

Test Cases

- TC01: Valid credentials $\rightarrow$ Success
- TC02: Invalid credentials $\rightarrow$ HTTP 401
- TC03: Five failures $\rightarrow$ Account Locked
- TC04: Empty Password $\rightarrow$ Validation Error

Conclusion: Specification-Driven Development (SDD) places the specification at the center of the software development lifecycle. In AI-assisted engineering, specifications become the source of truth because they are structured, version-controlled, reviewable, and less ambiguous than prompts. A complete specification should include intent, scope, constraints, interfaces, acceptance criteria, and test cases to ensure successful implementation and validation.